

Forecasting National Energy Profiles

Sparsh Sopory

*School of Computer Science and Information Technology,
Lucerne University of Applied Sciences and Arts, Lucerne, Switzerland*

Abstract—Forecasting how a country’s energy system will look several years out is useful for planning, yet most coursework stops at a notebook and a single accuracy number. This report describes an end-to-end machine-learning operations (MLOps) pipeline that turns a European energy forecasting model into a running, observable service. The data is drawn from the Our World in Data energy collection, cleaned down to 31 European countries and 18 indicators spanning 1990–2024 (1,083 country–year rows). A multi-output regressor predicts all 18 indicators at once from one- and two-year lags and their differences, trained on years up to 2019 and evaluated on 2020. Random Forest and Gradient Boosting variants both reach an aggregate R^2 of about 0.98 on the held-out year; we argue this figure is inflated by the strong year-to-year persistence of the series rather than by modelling skill, and we report it with that caveat. Around the model we build a Prefect flow that trains, batch-forecasts and checks for drift; an MLflow tracking store; a FastAPI service that forecasts recursively and logs every prediction to PostgreSQL; and a Grafana view over those logs. Drift is flagged only when a statistical signal coincides with a mean shift above 25%, which suppresses the false positives that small batches otherwise trigger. The whole stack runs under Docker Compose, with a `pytest` suite covering the data and drift logic.

Index Terms—MLOps, energy forecasting, multi-output regression, experiment tracking, model serving, data drift, Prefect, MLflow, FastAPI, Grafana.

I. INTRODUCTION

NATIONAL energy statistics change slowly. Population, gross domestic product and total energy consumption move by a few percent a year, while the generation mix shifts gradually as renewables displace fossil fuels. That predictability makes short-horizon forecasting tractable, and it is what this project exploits: given a country and a target year, predict its full energy profile, eighteen indicators at once, from population and GDP to the solar, wind and hydro electricity it generates.

The forecasting model, though, is the smaller half of the work. A model that lives only in a notebook is hard to trust, hard to rerun and impossible to watch over time. The remit of this project, set within an MLOps course, was to wrap a working forecaster in the machinery that production systems use: orchestrated training, tracked experiments, a served inference endpoint, persistent logging and a monitoring layer that notices when the inputs or outputs start to look unusual. This report documents that system. It describes the data and the cleaning behind it, the feature construction and the model,

and then, at greater length, the pipeline itself: a Prefect flow, an MLflow store, a FastAPI service backed by PostgreSQL, dashboards in Grafana, and a drift detector that pairs statistical tests with a guard against false alarms. Throughout we keep the reporting honest: the headline accuracy is high for reasons that have as much to do with the data as the model, and we say so.

II. BACKGROUND AND RELATED WORK

Energy-system indicators are highly autocorrelated time series. The single most informative predictor of next year’s value is usually this year’s value, which is why lag features are a natural and strong baseline for this kind of problem. We follow that intuition rather than reaching for a sequence model: tree ensembles over lagged features are cheap to train, robust to mixed feature scales, and easy to serve, which matters more here than squeezing out the last fraction of accuracy.

The operational side draws on now-standard MLOps building blocks. Experiment tracking with MLflow [6] records parameters, metrics and model artefacts so runs are comparable and reproducible. Workflow orchestration with Prefect [7] turns a sequence of scripts into a managed flow with retries and run history. Model serving uses FastAPI [8], and the predictions land in PostgreSQL [9] so they can be inspected and charted. For monitoring we lean on two classical tools for comparing distributions: the Kolmogorov–Smirnov (KS) two-sample test [13], available through SciPy [12], and the Population Stability Index (PSI) [14], borrowed from credit scoring, where it is a common rule-of-thumb measure of how far a distribution has moved. The dataset itself comes from Our World in Data [1].

III. DATA

The raw source is the Our World in Data energy dataset [1] (`owid-energy-data.csv`). A preprocessing script, written with pandas [2], narrows it to a clean, analysis-ready table in ten steps: it renames Czechia to Czech Republic for consistency, keeps a fixed list of 31 European countries, restricts the years to 1990–2025, and selects 20 columns (the country and year keys plus 18 numeric indicators). Rows with fewer than twelve non-missing values are dropped; the remaining gaps are forward-filled within each country, which is the right direction for a time series; duplicate country–year rows

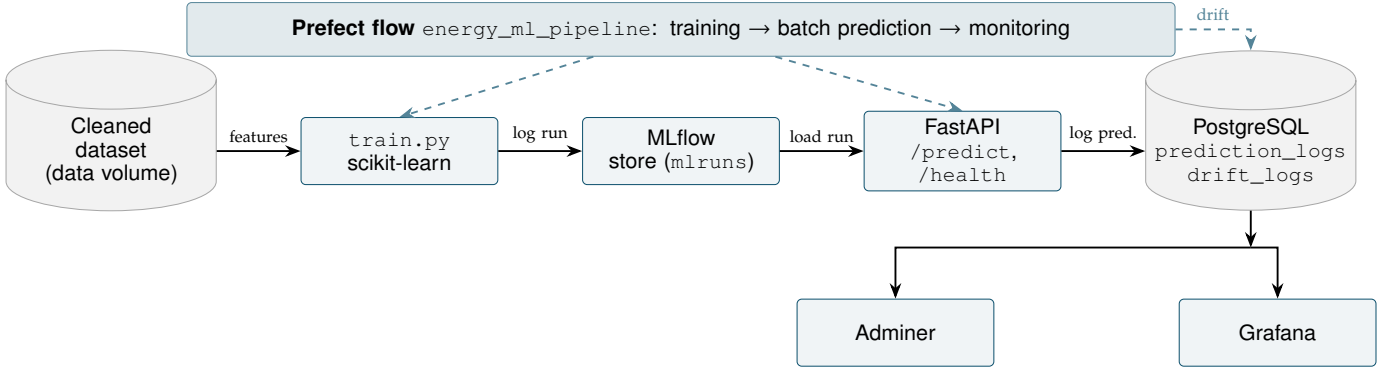


Fig. 1: System architecture. A Prefect flow (dashed control arrows) trains the model, drives batch forecasts through the FastAPI service, and runs a monitoring step. The API persists every prediction to PostgreSQL, which Adminer and Grafana read. The monitoring step compares those predictions against the same countries' recent real years from the dataset and writes a drift report back to the database. All components run as Docker Compose services.

are removed; and any country with fewer than ten years of data is discarded.

The result is 1,083 rows covering 31 countries from 1990 to 2024. Coverage is near-complete, 31 countries in every year from 1990 to 2023, dropping to 29 in 2024 because Iceland and Malta stop at 2023. The 18 indicators span four families: macro context (population, gdp), energy intensity (primary_energy_consumption, energy_per_capita, energy_per_gdp, electricity_demand, electricity_demand_per_capita), the fossil mix (fossil_fuel_consumption, fossil_electricity, fossil_share_energy) and the low-carbon mix (renewables, low-carbon, solar, wind and hydro electricity and their energy shares).

One detail matters for what follows. After the lag features are built (next section) and rows with missing lags are dropped, *Malta disappears from the modelling set*: five of its columns, fossil and renewables consumption and the fossil, renewables and low-carbon shares, are entirely empty, so every Malta row is removed. Thirty of the thirty-one countries therefore reach the model. This is a silent exclusion in the current code, and we return to it in the limitations.

IV. FORECASTING METHOD

A. Features

For each country c , year t and indicator f , the training script builds three derived features from the indicator's own recent history:

$$f_{c,t}^{(1)} = f_{c,t-1}, \quad f_{c,t}^{(2)} = f_{c,t-2}, \quad f_{c,t}^{\Delta} = f_{c,t}^{(1)} - f_{c,t}^{(2)}, \quad (1)$$

that is, the one- and two-year lags and their difference, computed strictly within each country so no series ever borrows from another. A scaled year, $\text{year} - \min(\text{year})$, is added as a mild time index. With 18 indicators this yields 54 lag/trend features; together with the country identity (one-hot encoded) and the scaled year, the input table has 56 columns, which the country one-hot expands further inside the pipeline, and the model predicts all

18 indicators at once. Rows without two prior years are dropped, leaving 969 modelling rows.

B. Models and evaluation

Two regressors from scikit-learn [3] are compared, each wrapped in a `MultiOutputRegressor` so a separate estimator is fit per target: a Random Forest [4] (250 trees, maximum depth 10, `random_state=42`) and Gradient Boosting [5] (150 estimators, learning rate 0.05). The split is temporal rather than random, training on all years up to and including 2019, and testing on 2020 alone, so the evaluation mimics genuine forecasting and cannot leak future information. Three metrics are logged: the coefficient of determination R^2 (averaged uniformly over the 18 targets), the mean absolute error (MAE, averaged over all targets and samples), and a normalised RMSE,

$$\text{NRMSE} = \frac{\text{RMSE}}{\max(Y) - \min(Y)}, \quad (2)$$

where the range in the denominator is taken over *all* targets jointly. We note in advance that this couples three quantities of very different magnitude (populations near 10^7 , GDP near 10^{12} , shares between 0 and 100), so the aggregate MAE and NRMSE are dominated by the largest-scale target, GDP. The R^2 , being unit-free, is the only one of the three that is comparable across indicators.

C. Recursive multi-year inference

At serving time the model is almost always asked for a year that lies beyond the end of the data, and a single forward pass cannot produce it: the features the model needs are the previous one and two years of every indicator, and for a future year those values simply do not exist yet. The service therefore forecasts *recursively*, one year at a time, feeding each year's prediction back in as the lag features for the next. Concretely, suppose the cleaned data ends in 2024 and a caller asks for 2030. The API does not jump straight to 2030; it first predicts 2025 from the real 2024 and 2023 values, then predicts 2026



Fig. 2: The three sequential tasks of `energy_ml_pipeline`.

using its own 2025 forecast (together with the real 2024) as the lags, then 2027 from the predicted 2026 and 2025, and so on, 2028, 2029, and finally 2030, each step consuming the outputs of the steps before it. The forecast for a distant year is therefore a chain of predictions stacked on predictions, built up year by year, rather than a direct leap from the last observed year.

This recursion is what lets the model reach arbitrarily far into the future from a fixed training window, but it carries an inherent cost: because every year after the first is conditioned on estimated rather than observed inputs, any error made early in the chain is carried forward and can amplify, so the forecasts grow more speculative the further out the horizon runs. The pipeline currently neither measures nor bounds this compounding, an asymmetry between training (on real lags) and inference (on the model's own outputs) that we return to in the limitations.

V. SYSTEM ARCHITECTURE AND IMPLEMENTATION

The system is a small set of cooperating services rather than a single program (Fig. 1). A Prefect flow orchestrates the offline work; an MLflow store keeps the trained models and their metrics; a FastAPI service answers forecast requests and writes them to PostgreSQL; and Adminer and Grafana read that database for inspection and dashboards. Everything is packaged as Docker Compose [11] services so the stack comes up with a single command.

A. Orchestration with Prefect

The flow `energy_ml_pipeline` chains three tasks (Fig. 2). `training_task` runs the training script as a subprocess, parses the best model from its output and attaches a Markdown summary to the run. `batch_prediction_task` invents a small workload, one randomly chosen European country across five consecutive future years, and posts each request to the running API. `monitoring_task` then loads the most recent logged predictions, which across successive runs can span more than one country, and checks them for drift. The flow is exposed as a Prefect deployment named `energy-pipeline` via `.serve()`. Figure 3 shows two completed runs of this deployment, each executing all three tasks, with wall-clock times of about six and three and a half minutes.

B. Tracking with MLflow

Each training run is logged to a local MLflow store under the experiment `Energy_Prediction_Model`,

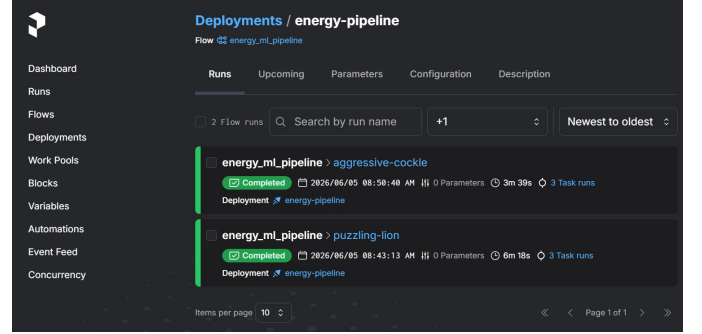


Fig. 3: Prefect deployment `energy-pipeline` for the flow `energy_ml_pipeline`. Two completed runs are shown, each with three task runs; wall-clock times 6 m 18 s and 3 m 39 s.

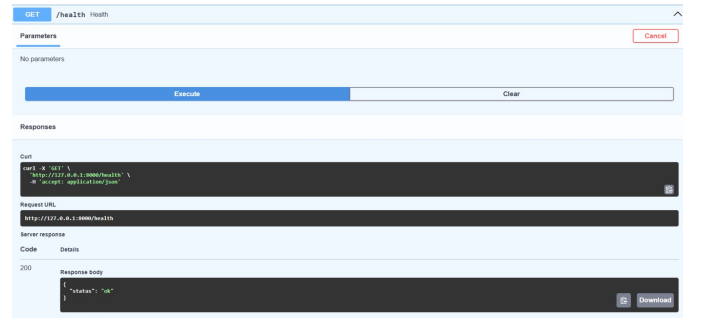


Fig. 4: The FastAPI GET `/health` check through the Swagger UI, returning `{status: ok}`.

recording the model type as a parameter and R^2 , MAE and NRMSE as metrics, alongside the fitted pipeline as an artefact. This is what makes the two models comparable in Section VII, and it is how the served model is identified: the API loads a fixed run by id.

C. Serving with FastAPI

The inference service exposes three routes: a root status route, a GET `/health` check, and POST `/predict`, which takes a country and a target year. The service was deliberately written to be testable and portable: the model load and the database connection are both lazy and can be switched off through environment variables (`SKIP_MODEL_LOAD`, `DISABLE_DB_LOGGING`), and all paths and credentials come from the environment with local defaults, so there are no hard-coded machine-specific paths. Figures 4 and 5 show both endpoints responding; the `/predict` call for Germany in 2027 returns all 18 indicators, and that exact response is the row that later appears in the prediction log (Fig. 6, id 22).

D. Logging and inspection

Every successful prediction is written to the `prediction_logs` table with its country, year, the 18 predicted values and the request latency. Figure 6

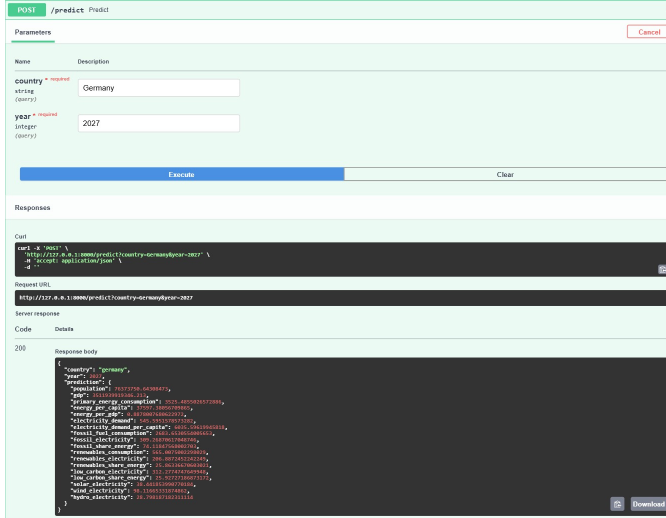


Fig. 5: The FastAPI POST /predict endpoint through the Swagger UI. The forecast for Germany in 2027, population ≈ 76.4 M, GDP $\approx 3.51 \times 10^{12}$, renewables share $\approx 25.9\%$, fossil share $\approx 74.1\%$, returns all 18 indicators and is persisted to PostgreSQL, reappearing as id 22 in Fig. 6.

shows the table in Adminer with 22 logged forecasts across five countries. The companion `drift_logs` table, created idempotently by the monitoring task, holds the drift reports discussed in the next section.

E. Testing and tooling

The repository ships a `pytest` suite that pins the logic most likely to break silently. Tests cover the preprocessing rules (the European filter, the Czechia rename, the year range, the ten-year minimum, de-duplication and the final column set), the feature engineering (that lags shift correctly, that `trend` equals `lag1` minus `lag2`, that the scaled year starts at zero, and crucially that no country’s lags ever leak from another), and the drift helpers (KS returning zero for identical inputs and significance for shifted ones, PSI near zero for identical distributions and large for shifted ones, and sensible behaviour on empty or constant inputs). The FastAPI integration tests run against a tiny fixture dataset and a stub model, and are gated behind a flag so the rest of the suite passes before that wiring is in place. Linting and formatting use `Ruff`, and coverage is configured over the two source packages, all declared in `pyproject.toml`.

VI. MONITORING AND DRIFT DETECTION

Monitoring asks a blunt question after each batch: do the freshly produced forecasts look like the recent past? The monitoring task loads the most recent batch from `prediction_logs` and, for the same countries, the real values from 2020–2024 in the cleaned dataset. For every indicator it computes two distance measures between the

Fig. 6: Adminer view of `prediction_logs` (22 rows). Each row is one API forecast, country, year, the 18 predicted indicators and request latency, here covering Austria, Sweden, Finland, the United Kingdom and Germany.

reference (real) and current (forecast) distributions. The first is the KS statistic [12], [13],

$$D = \sup_x |F_{\text{ref}}(x) - F_{\text{cur}}(x)|, \quad (3)$$

with its p -value. The second is the PSI [14] over quantile bins of the reference,

$$\text{PSI} = \sum_b (p_b^{\text{cur}} - p_b^{\text{ref}}) \ln \frac{p_b^{\text{cur}}}{p_b^{\text{ref}}}. \quad (4)$$

A naive rule would flag an indicator whenever either test fires. That fails badly here, because each side of the comparison has only about five to ten points: with samples that small the KS test bottoms out at its minimum p -value and the quantile-based PSI inflates wildly, so almost everything looks “drifted” even when nothing meaningful has changed. The pipeline guards against this with a conjunctive rule, an indicator is flagged only if a statistical signal *and* a real move in the mean both hold:

$$\begin{aligned} \text{drift} &\iff (p < 0.05 \vee \text{PSI} \geq 0.10) \\ &\wedge \frac{|\mu_{\text{cur}} - \mu_{\text{ref}}|}{|\mu_{\text{ref}}|} > 0.25. \end{aligned} \quad (5)$$

Table I shows the rule at work on one batch (Austria and Sweden, ten reference points against six forecasts). Every PSI exceeds 5, far above the 0.25 “major shift” rule of thumb, so the statistical arm of the rule is always satisfied; it is the 25% mean-change guard that does the discriminating. Seven of the eighteen indicators clear it, and they are exactly the volatile generation terms: low-carbon electricity more than triples, solar and wind roughly double, fossil electricity more than halves, and renewable consumption, renewable generation and hydro each move by a quarter to a third. Stable quantities such as energy per capita (-1.4%) or the renewables share ($+6.2\%$) are not flagged, despite their large PSI. The full set of reports, 72 rows, four batches of eighteen indicators, is in Fig. 7, and the forecast trajectories that feed this comparison can be browsed in Grafana (Fig. 8).

TABLE I: Drift report for the `austria_sweden` batch (10 reference vs. 6 forecast points). $\Delta\bar{x}$ is the relative change in the indicator mean, computed from the logged reference/current means; *Drift* applies the rule (statistical signal and $|\Delta\bar{x}| > 25\%$).

Indicator	KS	p	PSI	$\Delta\bar{x}$	Drift
population	0.83	0.004	10.80	+7.9%	no
gdp	0.83	0.004	5.23	+12.3%	no
prim. energy cons.	0.63	0.066	10.80	+15.0%	no
energy_per_capita	0.50	0.249	10.80	-1.4%	no
energy_per_gdp	0.50	0.249	12.20	-0.6%	no
electricity_demand	0.50	0.249	10.80	+15.8%	no
elec. dem./cap.	0.73	0.019	10.80	+17.9%	no
fossil_fuel_cons.	0.83	0.004	10.80	-20.5%	no
fossil_electricity	0.53	0.170	7.65	-56.5%	yes
fossil_share_energy	0.50	0.249	10.80	-20.1%	no
renewables_cons.	0.73	0.019	10.80	+34.7%	yes
renewables_elec.	0.50	0.249	9.02	+25.7%	yes
renew. share_energy	0.50	0.249	10.80	+6.2%	no
low_carbon_elec.	0.83	0.004	10.80	+233.3%	yes
low_carbon_share	0.50	0.249	10.80	+16.6%	no
solar_electricity	0.80	0.009	10.80	+142.4%	yes
wind_electricity	0.83	0.004	10.80	+93.8%	yes
hydro_electricity	0.63	0.066	9.20	+28.1%	yes

Fig. 7: Adminer view of `drift_logs` (72 rows: four batches $\times$ 18 indicators). Columns record the KS statistic, p -value, PSI, the drift flag, reference and current means, sample sizes, and a `source` label naming the batch’s countries. Table I summarises the `austria_sweden` batch (top eighteen rows).

VII. RESULTS

A. Experimental setup

The reported numbers come from the temporal split of Section IV-B: training on 820 rows spanning 1992–2019 across the 30 modelled countries, and testing on the 30 rows of 2020. The remaining 119 modelling rows, those for 2021–2024, fall outside the $\leq 2019/ = 2020$ split and enter neither set. Training runs are inexpensive, about 48.5 seconds for a Gradient Boosting run on a local MLflow store (Fig. 10, tracking server 3.10.0). All metrics are the multi-target aggregates defined earlier.

B. Model comparison

Both models fit the held-out year extremely well on the unit-free metric and are indistinguishable on it:

TABLE II: Model comparison on the 2020 hold-out (aggregate metrics over 18 indicators). Bar charts in Fig. 9 show all runs cluster at $R^2 \approx 0.98$ and $\text{NRMSE} \approx 0.01$; Gradient Boosting values are from the run detailed in Fig. 10.

Model	R^2	MAE	NRMSE
Random Forest	≈ 0.98	3.08×10^9	≈ 0.01
Gradient Boosting	0.9817	3.46×10^9	0.0089

across every logged run, $R^2 \approx 0.98$ and $\text{NRMSE} \approx 0.01$ (Fig. 9a,b). They separate only on MAE (Fig. 9c): every Random Forest run lands at the same 3.08×10^9 , while Gradient Boosting runs sit higher, between 3.31 and 3.46×10^9 . The Random Forest figure is identical run to run because its `random_state` is fixed; Gradient Boosting sets no seed, which is why its runs vary slightly. Table II collects the figures, taking the Gradient Boosting values from the detailed run in Fig. 10.

C. Forecasts and end-to-end operation

Beyond the metrics, the pipeline produces and stores plausible forecasts. The live `/predict` call for Germany in 2027 (Fig. 5) returns a coherent profile, population about 76.4 million, GDP about 3.51×10^{12} , a renewables share near 26% against a fossil share near 74%, and that same record is the one logged at `id 22` in Fig. 6, confirming the request-to-database path end to end. The Grafana panel in Fig. 8 then renders such forecasts over time straight from the log. On the monitoring side, the four batches in `drift_logs` (Fig. 7) show the detector running on every flow execution and flagging only the indicators that genuinely move, as summarised in Table I.

VIII. DISCUSSION

The most important thing to say about the R^2 of 0.98 is that it is not as impressive as it looks. With the previous year’s value handed to the model as a feature, and with energy indicators that change by only a few percent a year, even a trivial “predict last year again” rule would score highly. The number mostly reflects the autocorrelation of the data, not the cleverness of the ensemble. The project does not include such a baseline, so we cannot quantify how much the models add over it, an omission worth fixing.

The aggregate MAE and NRMSE deserve similar care. Reported in the billions, the MAE is essentially the GDP error: averaging absolute errors across eighteen targets of wildly different scale lets the 10^{12} -scale GDP dominate while the percentage shares contribute almost nothing. The NRMSE, normalised by the single widest target range, is GDP-scaled for the same reason. These are fine as relative comparisons between two models on the same footing, but they say little about how well any individual indicator is predicted. Per-indicator metrics would be far more informative, and indeed the Grafana

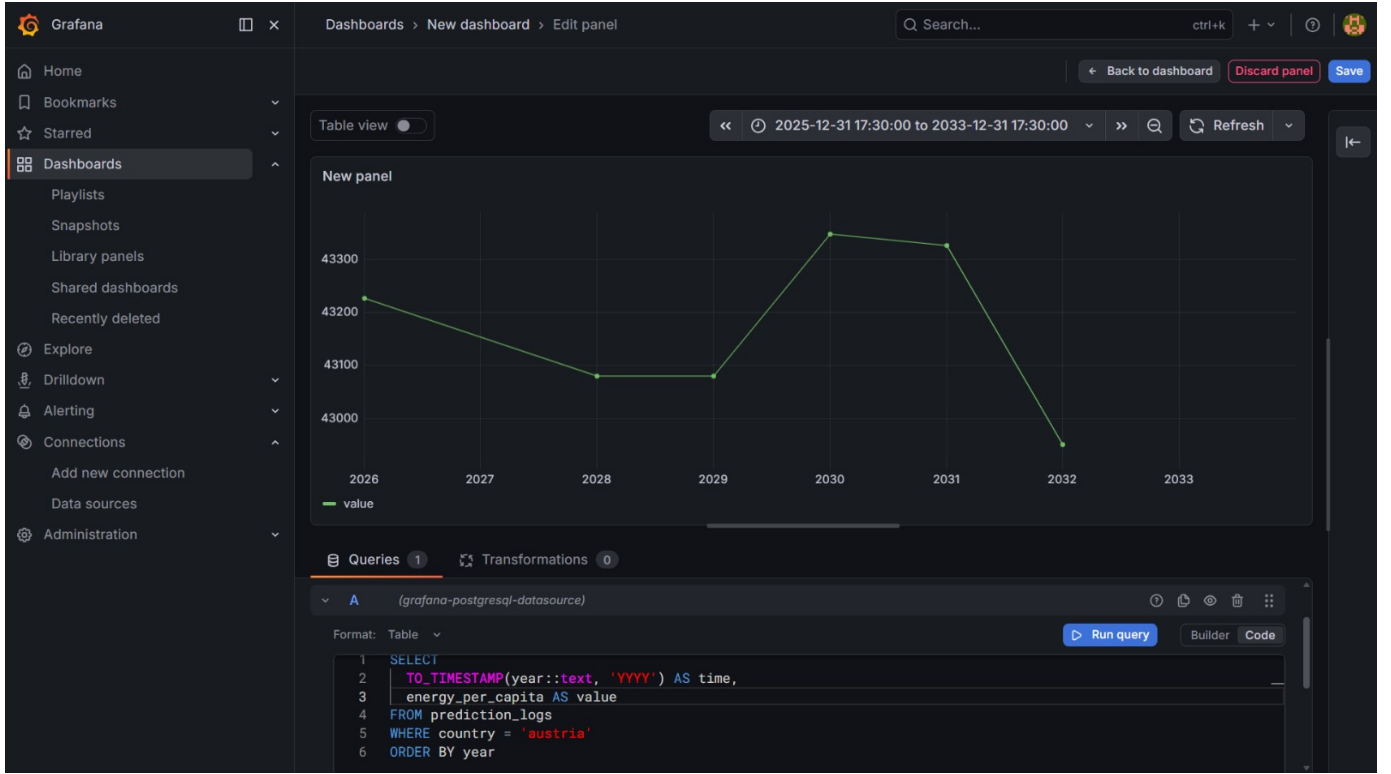


Fig. 8: A Grafana time-series panel backed by the PostgreSQL datasource, reading `prediction_logs` directly. It plots the forecast trajectory of `energy_per_capita` for Austria across the 2026–2032 horizon. The provisioned dashboard JSON in the repository defines a fuller set of monitoring panels [10].

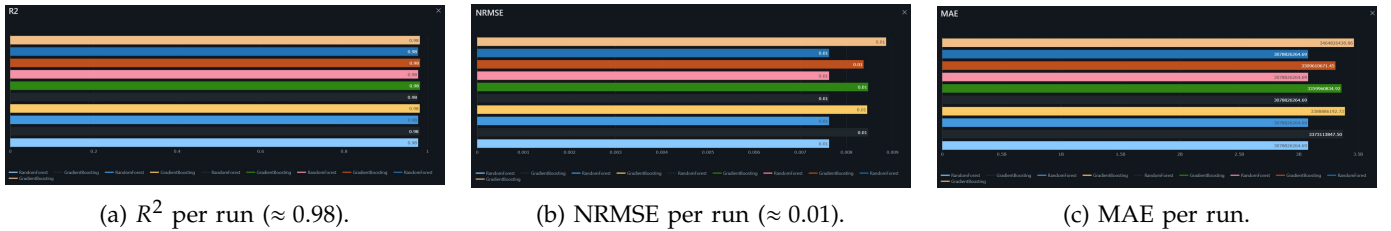


Fig. 9: MLflow metric comparison across all logged runs of the `Energy_Prediction_Model` experiment. The models tie on R^2 and NRMSE and separate on MAE, where Random Forest (lower, $\approx 3.08 \times 10^9$) edges Gradient Boosting ($3.31\text{--}3.46 \times 10^9$).

layout anticipates them, but the current code does not yet compute them.

The drift results are best read as a demonstration of the guard rather than as evidence of model failure. Because each batch contains a handful of points, and because the forecasts deliberately reach into years beyond the reference window, the distributions differ for entirely expected reasons; the PSI and KS signals therefore fire almost everywhere. What the 25% mean-change condition adds is restraint, it converts a wall of false alarms into a short, interpretable list of indicators that have actually shifted. That said, this is closer to monitoring forecast magnitudes than to detecting covariate drift in the usual train-versus-serve sense, since the comparison is forecast-versus-recent-history rather than against the training distribution.

IX. LIMITATIONS AND FUTURE WORK

Several limitations follow directly from the design and should temper any reading of the results. The model is evaluated on a single year, 2020, with one observation per country, thirty points in total, so the accuracy estimate is fragile; a rolling-origin backtest across several test years would be far more trustworthy. Multi-year forecasts compound error, because the model is trained on real lags but, at inference, consumes its own predictions as lags; the further out the horizon, the more speculative the output, and the pipeline currently neither measures nor bounds this growth. Malta is dropped from the modelling set without comment, owing to columns that are entirely missing; it should be handled explicitly, whether by imputation or by modelling it separately. And, as discussed, the absence of a persistence or linear

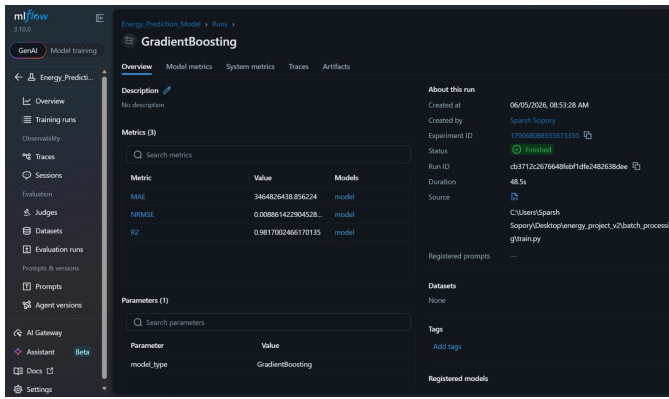


Fig. 10: MLflow run overview for one Gradient Boosting run (model_type=GradientBoosting): aggregate $R^2 = 0.9817$, $MAE = 3.46 \times 10^9$, $NRMSE = 0.0089$, trained in 48.5s; tracking server 3.10.0.

baseline leaves the models' real contribution unquantified. Each indicator is, moreover, modelled independently inside the `MultiOutputRegressor`, so strong cross-indicator relationships, renewables consumption and the renewables share move together, for instance, are never exploited; a regressor chain or a genuinely joint multi-target model could enforce that coherence.

There is also engineering to tidy. Gradient Boosting should be given a fixed seed for reproducibility. The served model is pinned by a hard-coded run id through an environment variable; promoting the best model through the MLflow Model Registry would be cleaner and would decouple serving from a specific run. The database password is a default and belongs in a secret. The per-indicator backtest metrics that the dashboard expects should be computed and written so that view is populated. Finally, the flow is run on demand through `.serve()`; attaching a schedule would turn it into a genuinely autonomous pipeline.

X. CONCLUSION

This project takes a modest forecasting task, predicting eighteen European energy indicators a few years ahead, and builds the operational scaffolding that makes it trustworthy to run and to watch. Lagged tree ensembles fit the held-out year with a high aggregate R^2 , though we are careful to attribute most of that to the data's persistence rather than to the model. The substance is the pipeline around it: orchestrated training and monitoring in Prefect, tracked experiments in MLflow, a tested FastAPI service that logs every forecast to PostgreSQL, dashboards in Grafana, and a drift detector whose conjunctive rule keeps small-sample false alarms in check. The result is a small but complete MLOps system, with its weaknesses, a single evaluation year, scale-dominated aggregate metrics, compounding multi-year forecasts, named plainly rather than hidden.

REPRODUCIBILITY AND ARTIFACT AVAILABILITY

The source code is available at https://github.com/ss0109/Energy_Proj, and a prebuilt container image is published to the GitHub Container Registry, ghcr.io/ss0109/energy_proj. The full stack, API, PostgreSQL, Adminer and Grafana, comes up through Docker Compose; the database schema and image build are provided in the repository (db/init.sql and the Dockerfile referenced by the compose file). The served model is selected by the `MLFLOW_RUN_ID` environment variable (default `7c855f5fcf5a4a6193976fa7a5a4aa89`), and the API's data path and database credentials are likewise environment-driven, so the service runs without machine-specific configuration. Unit tests run with `pytest`; the FastAPI integration tests are enabled through an environment flag once the service is wired to the test fixtures.

GENERATIVE AI DECLARATION

Generative AI tools were used to assist with drafting and editing the language of this report and with formatting figures and tables. All experiments, code, data processing, results and figures are the author's own; every reported number was taken from the project's own runs and artefacts, and the author reviewed and is responsible for the final content.

REFERENCES

- [1] H. Ritchie, P. Rosado, and M. Roser, "Energy," *Our World in Data*, 2023. [Online]. Available: <https://ourworldindata.org/energy> (dataset: `owid-energy-data`).
- [2] W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Science Conf.*, 2010, pp. 56–61.
- [3] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [4] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [5] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.
- [6] M. Zaharia et al., "Accelerating the machine learning lifecycle with MLflow," *IEEE Data Eng. Bull.*, vol. 41, no. 4, pp. 39–45, 2018.
- [7] Prefect Technologies, "Prefect documentation," 2024. [Online]. Available: <https://docs.prefect.io>
- [8] S. Ramírez, "FastAPI," 2024. [Online]. Available: <https://fastapi.tiangolo.com>
- [9] The PostgreSQL Global Development Group, "PostgreSQL documentation," 2024. [Online]. Available: <https://www.postgresql.org/docs>
- [10] Grafana Labs, "Grafana documentation," 2024. [Online]. Available: <https://grafana.com/docs>
- [11] Docker Inc., "Docker documentation," 2024. [Online]. Available: <https://docs.docker.com>
- [12] P. Virtanen et al., "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.
- [13] F. J. Massey, "The Kolmogorov-Smirnov test for goodness of fit," *J. Amer. Statist. Assoc.*, vol. 46, no. 253, pp. 68–78, 1951.
- [14] B. Yurdakul, "Statistical properties of population stability index," Ph.D. dissertation, Western Michigan Univ., 2018.